



**We provide Accelerated AI Solutions through Vertical Integration
of Co-optimized Algorithm, SW and HW Technologies.**

Introduction to NPU and SDK

**Powerful Hardware Better Intelligence**

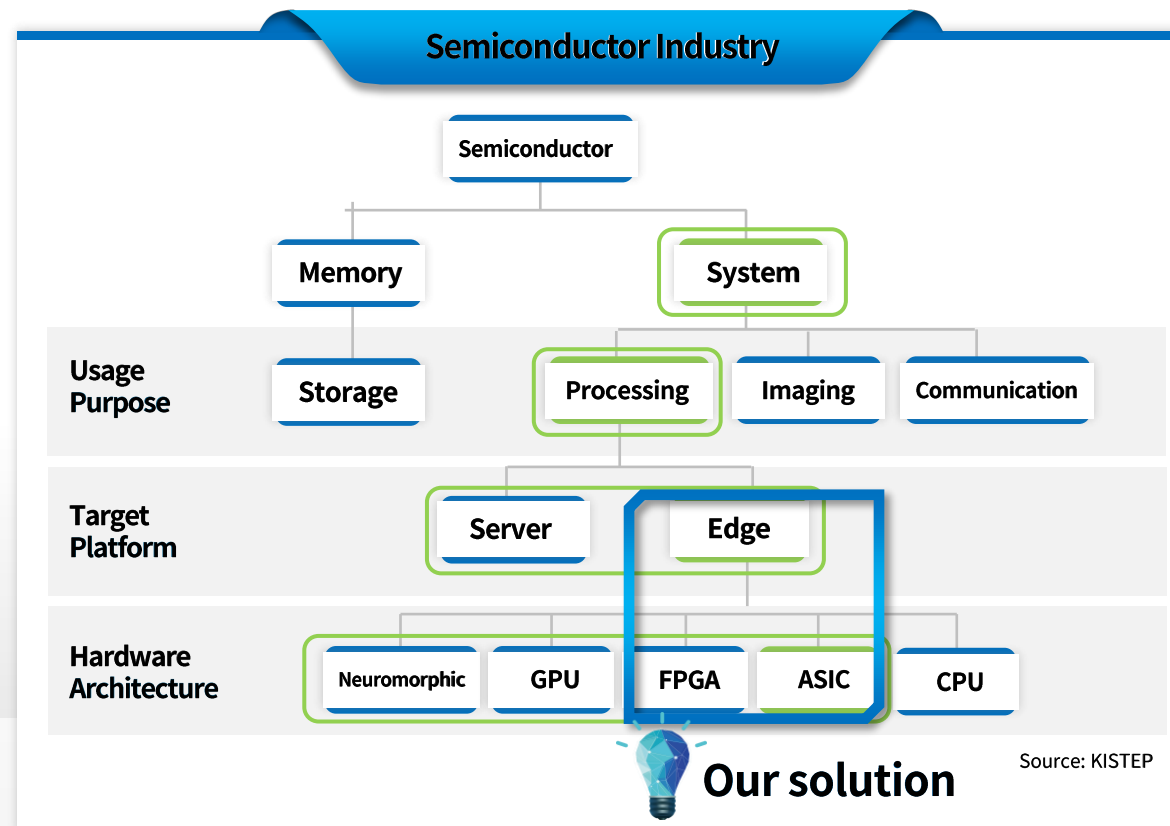
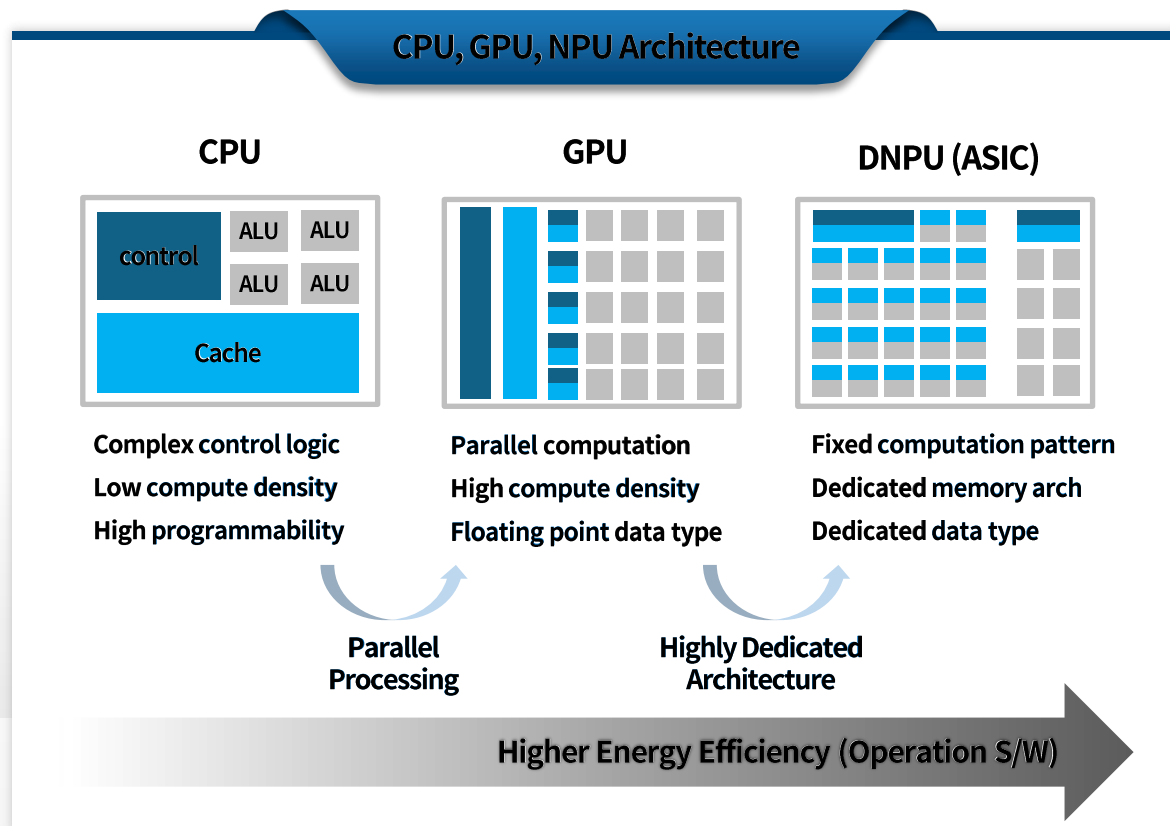
모빌린트는 최첨단 지능형 반도체를 통해 스마트한 세상을 만듭니다.

| MOBILINT History

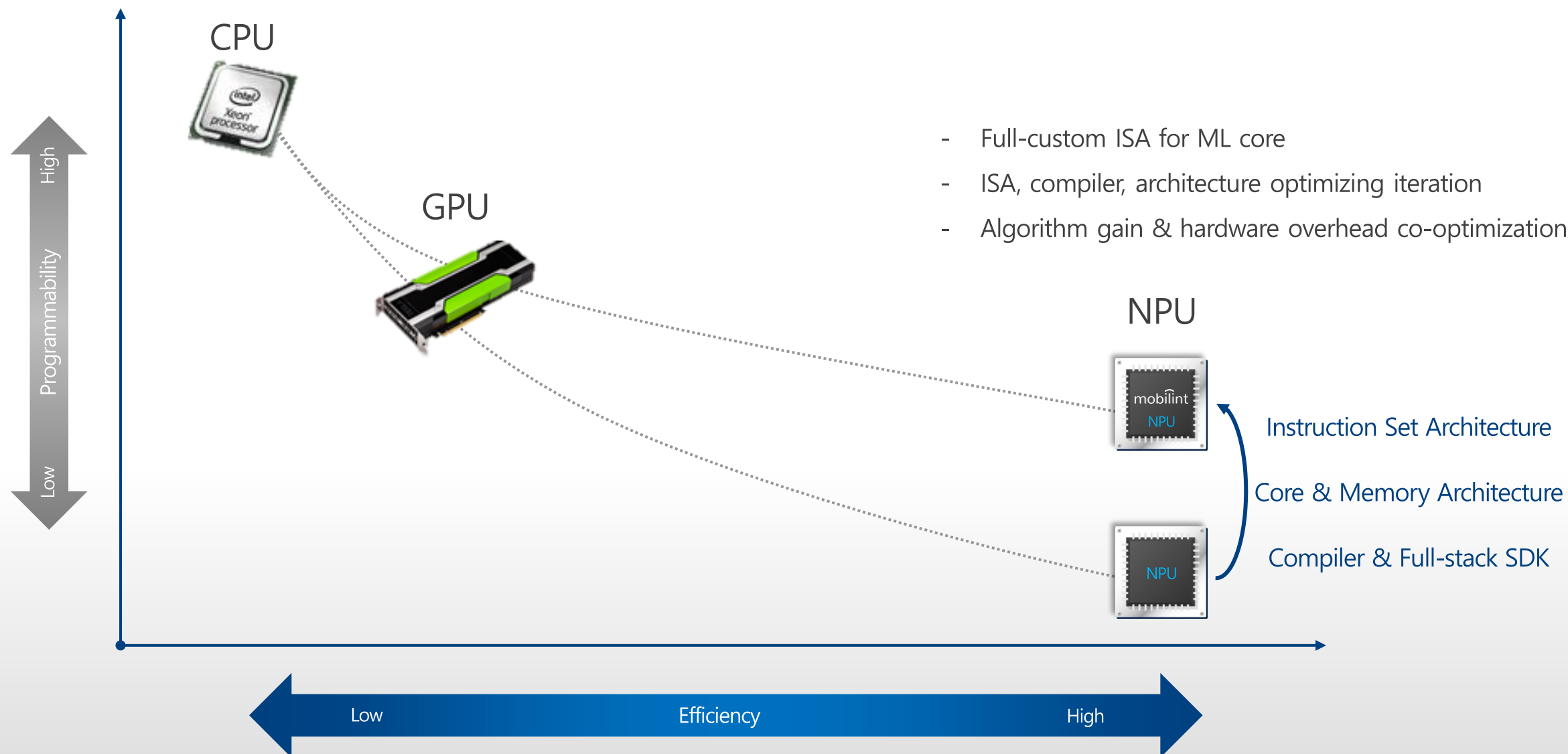


회사명	주식회사모빌린트	대표자	신동주
설립일	2019.04.17.	사업자등록번호	876-88-01379
업종	반도체칩,반도체 시스템 설계 및 연구개발업 / 시스템,응용 소프트웨어 개발 및 공급업		
주요제품	딥러닝 가속 & 센서 데이터 처리를 위한 NPU Solution		
소재지	서울특별시 강남구 선릉로93길 35, 3층		

지능형 반도체(AI Semiconductor or NPU)는 딥러닝(AI기법) 연산에 최적화된 프로세서로
동시다발적인 연산을 효율적으로 처리하는 반도체



Programmability & Efficiency Trade-off



ARIES High-Performance AI Accelerator



Target Application

Large Devices or On-Premises AI

Performance

64 TOPS (Boost 80 TOPS)

Host Interface

PCIe Gen 4.0 8-Lane

Power (TDP)

25W

REGULUS Standalone AI SoC



Target Application

On-Device AI

Performance

8 TOPS (Boost 10 TOPS)

Interface

MIPI, USB3.1, Ethernet, etc.

Power (TDP)

<5W

Intelligence Everywhere

MOBILINT ARIES Specifications



Product Type (Aries Embedded)



PCIe Card – MLA100



AI Box – MLX-A1

Peak Performance (INT8)	64 TOPS (Boost 80 TOPS)
Host Interface	PCIe Gen 4.0 8-lane
Supported Frameworks	TensorFlow, Pytorch, ONNX, Keras, and TVM
Supported Host Hardware	X86 and ARM AArch64 based architecture
DRAM Type	4CH 32-bit wide LPDDR4/LPDDR4X
DRAM Speed	4266 Mbps

SAMPLE AVAILABLE NOW

World Best Price

Offers a superlative hardware upgrade with competitive cost

Easy and Fast to Use

Accommodates a full-stack SDK that natively supports TensorFlow, PyTorch, ONNX, Keras, and TVM

Programmability

Supports 50+ DNN models while maintaining stellar performance

Accelerated Quantization

Achieves FP32 accuracy and best efficiency with PTQ & QAT

Versatile Performance

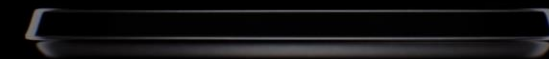
Facilitates scalability and versatility by implementing effective scheduling and multitasking features

Green Digital Block

Retains energy efficiency with high data reusability and memory access proficiency innovations

MOBILINT ARIES

A CHIP HERE TO VENTURE YOUR VISION FOR THE FUTURE



MOBILINT ARIES

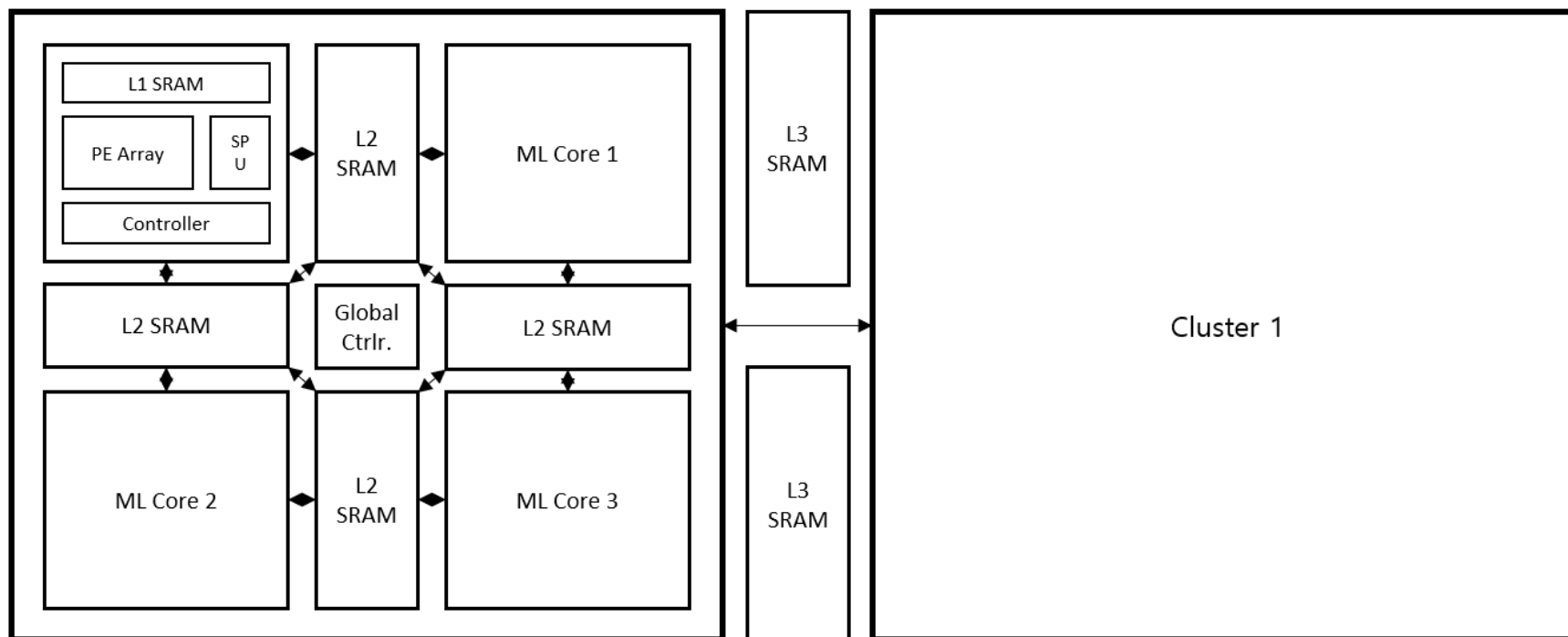
Deep Learning Accelerator



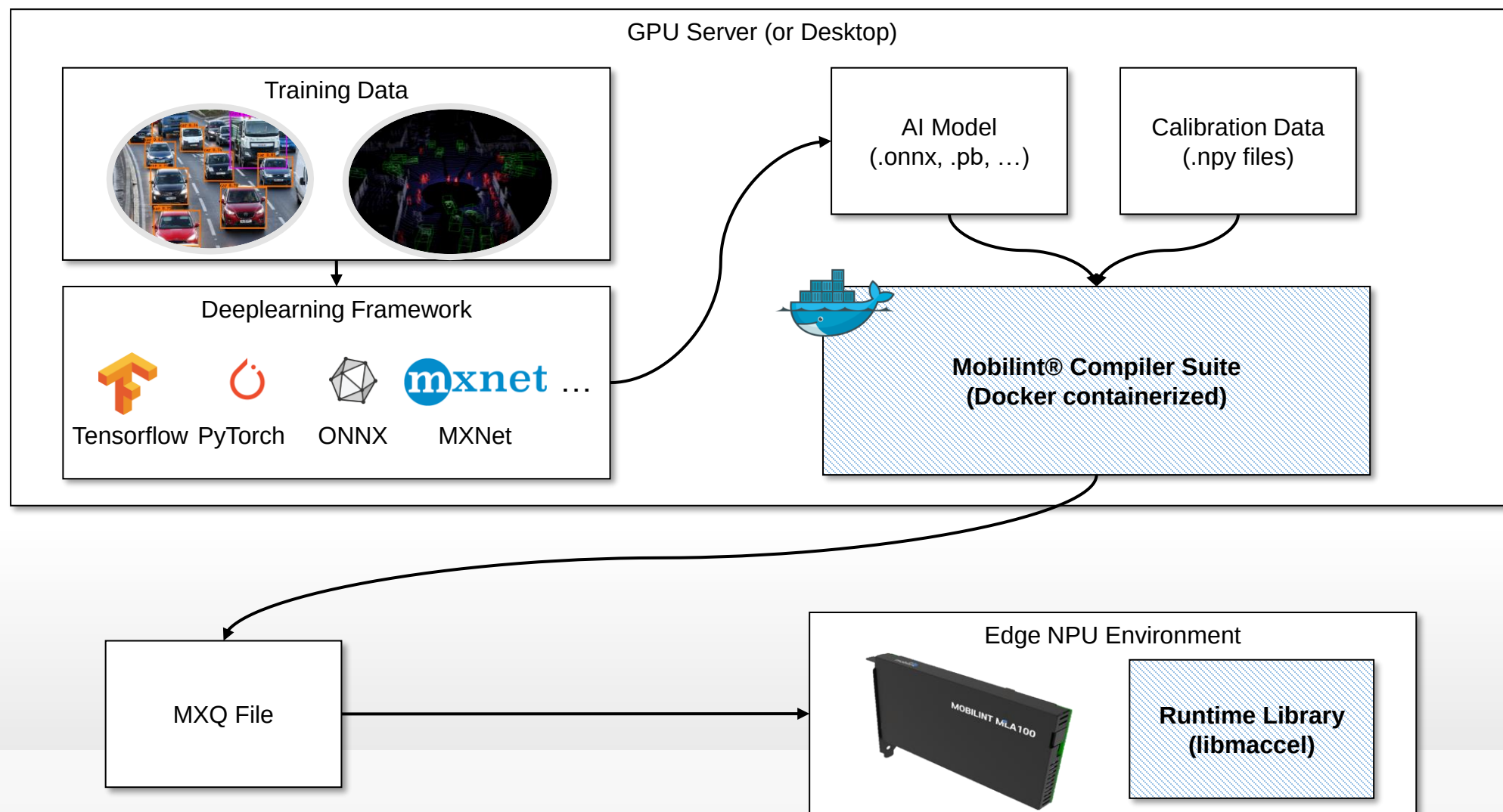
Sample Available NOW

- Multi-core/Multi-cluster: Area Overhead for Control Units

- 64 MAC/cycle ~ 256K MAC/cycle
- Single Core (64 MACs, 128 MACs, 256 MACs, 512 MACs, 1K MACs, 2K MACs, 4K MACs, 8K MACs, 16K MACs)/cycle
- Multi-Core (1~4 cores, 4 cores in aries)
- Multi-Cluster (1~4 clusters, 2 cluster in aries)

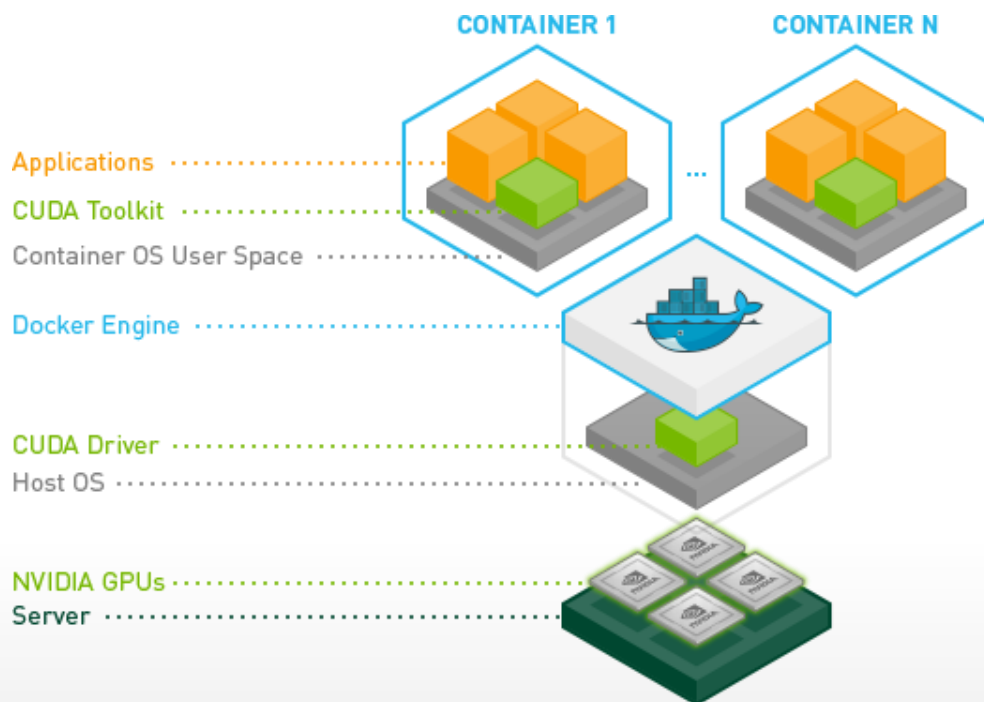


Development Workflow

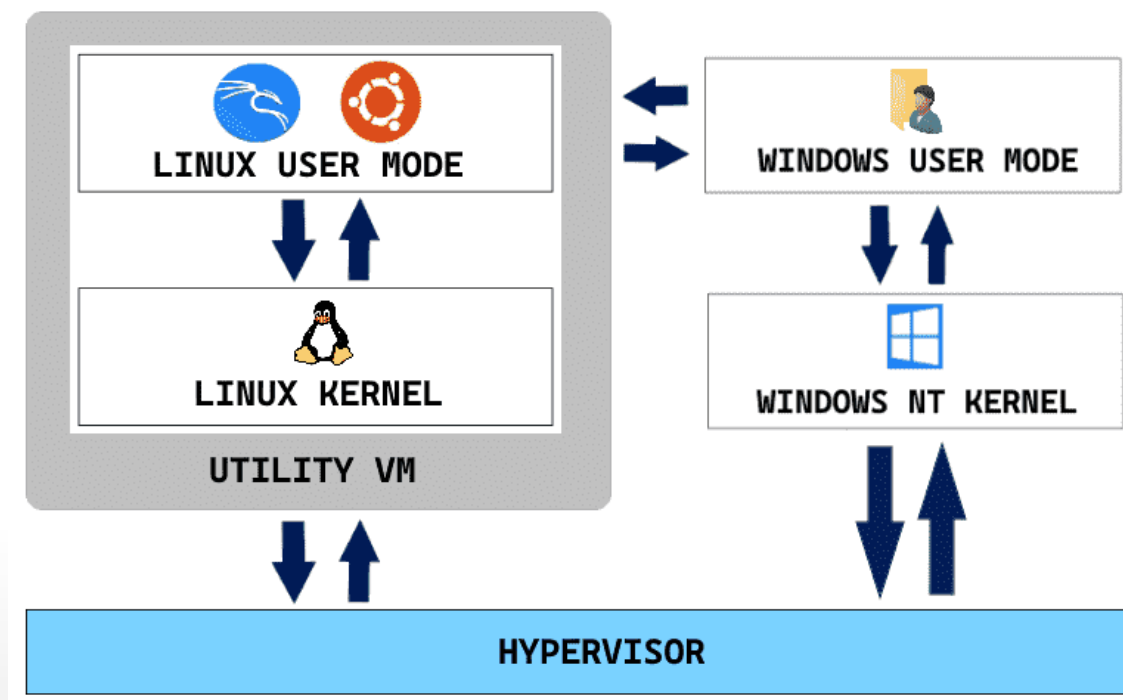


Pre-request for compiler

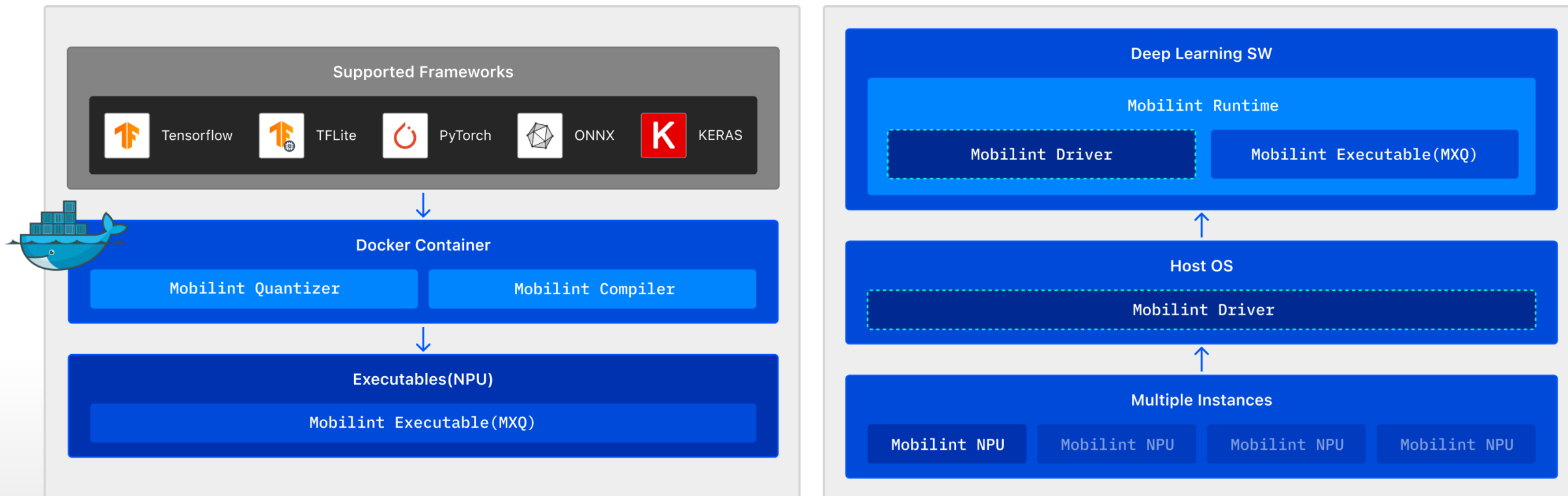
- ✓ Docker (essential)
NVIDIA Docker (recommended)



- ✓ (Windows) WSL2



SDK Components



Step0. install mobilint docker

✓ Docker hub을 통해 mobilint/qbcompiler 도커 이미지 다운로드

- GPU: mobilint/qbcompiler:v0.9.0.2
- CPU: mobilint/qbcompiler:v0.9.0.2-cpu

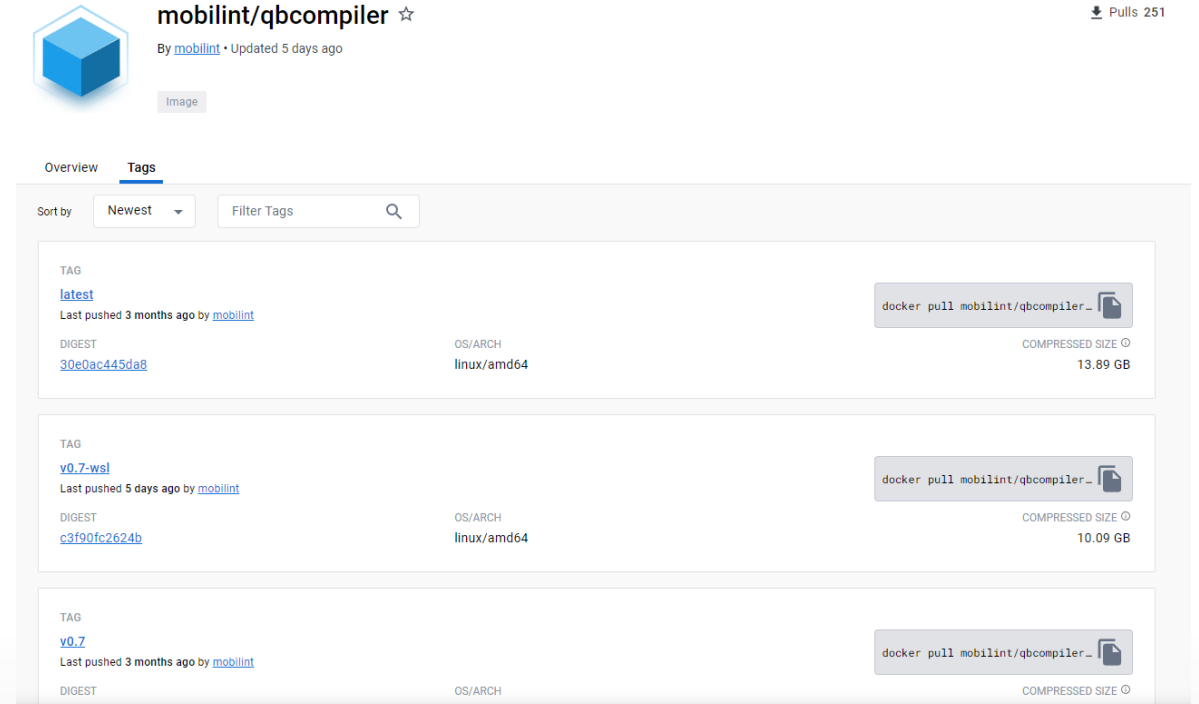
✓ 도커 이미지 설치 명령어

→ `docker pull mobilint/qbcompiler:v0.9.0.2`

✓ 도커 컨테이너 설치 명령어 (예시)

→ `docker run --gpus all -it -v /path/to/workspace:/workspace --name qbcompiler mobilint/qbcompiler:v0.9.0.2 /bin/bash`

* docker 내부에서 NPU 사용을 원하는 경우, driver 설치된 상태에서 `--device /dev/aries0:/dev/aries0` 을 추가



Step1. Download Compiler

☑ 모빌린트 배포사이트를 통해 SDK 및 매뉴얼 다운로드 (<https://dl.mobilint.com/signin.html>)

회원명 / User Info / Admin / Logout

Mobilint® Software Development Kit

Welcome!



This is the page provided to customers for downloading the Mobilint Software Development Kit (SDK). This page is designed to provide downloads for the Compiler, Runtime Library, and NPU Driver. Additionally, it is followed by an Additional Resources section that includes necessary manuals and other materials for utilizing the SDK.

We sincerely appreciate your consideration of Mobilint's product offerings. If you have any inquiries or questions, please do not hesitate to [contact us here](#).

[Regulus](#) [Aries](#) [Aries 2](#)

The Model Compiler

The compiler converts models that users are already proficiently using (such as PyTorch, TensorFlow, ONNX, etc.) into a format executable on the Mobilint NPU. It incorporates state-of-the-art techniques from recent papers, such as high-performance quantization, deep learning model optimization, and NPU optimization. This SDK component represents the essence of Mobilint's deep learning technology.

Below is the compiler image provided to customers. In most cases, it is advisable to choose the latest version.

qbcompiler:v0.9.0.2-cpu
v0.9.0.2-aries2-ubuntu-22.04-5

HW별 Tab Click 후 다운로드

Runtime Library

The runtime library facilitates the easier and faster application of compiled models on the NPU within the application. Mobilint, which focuses on edge-oriented NPUs, has embedded system optimization skills in the runtime library to ensure that models perform optimally even in challenging and unpredictable environments. By utilizing the runtime library, users can develop programs more conveniently compared to directly handling the NPU from scratch, while achieving similar or even better performance.

Below is the runtime library provided to Mobilint. In most cases, it is advisable to choose the latest version. Mobilint supports C++ and Python, and all of these are included in the distribution packet.

IMPORTANT! It is essential to ensure that the driver is installed before using the runtime library.

Ubuntu 22.04.3 LTS	
Intel x86-64	v0.12.0-py3.10.12

NPU Driver

The driver is a module that enables the NPU to be functional within the system. To utilize the Mobilint NPU, its installation is **MANDATORY**.

Below is the NPU driver provided to Mobilint. In most cases, it is advisable to choose the latest version.

All Linux	
Aries	v0.1.0

Additional Resources

Below is the additional materials provided to Mobilint.

#	Type	Name	Date
1	Compiler	Compiler Manual (v0.7.11)	2023-09-01 15:20:56
2	Runtime Library	Runtime Library Manual (v0.12.0-all)	2023-09-01 15:22:30
3	Etc	TestDrive Setup Package (v1.0.0)	2023-09-07 18:51:47

Step2. Install Compiler

✓ Docker Container에 접속

- docker start qbcompiler
- docker exec -it qbcompiler /bin/bash

```
mobilint@DESKTOP-BN80M01:~/workspace$ docker start qbcompiler
qbcompiler
mobilint@DESKTOP-BN80M01:~/workspace$ docker exec -it qbcompiler /bin/bash
root@42fe959f7443: /#
```

✓ Install compiler

pip install [컴파일러 whl 파일 이름]

```
root@42fe959f7443: /workspace# pip install qubee-0.7.12-py3-none-any.whl
Processing ./qubee-0.7.12-py3-none-any.whl
Requirement already satisfied: onnx==1.12.0 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (1.12.0)
Requirement already satisfied: pycocotools in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (2.0.7)
Requirement already satisfied: onnxruntime-gpu==1.14.1 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (1.14.1)
Requirement already satisfied: torch>=1.10.1 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (2.0.1)
Requirement already satisfied: msgpack in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (1.0.5)
Requirement already satisfied: tensorflow==2.9 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (2.9.0)
Requirement already satisfied: torchvision==0.11.2 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (0.11.2)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (6.0.1)
Requirement already satisfied: onnxruntime==1.14.1 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (1.14.1)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.8/dist-packages (from qubee==0.7.12) (1.24.0)
Requirement already satisfied: protobuf<=3.20.1,>=3.12.2 in /usr/local/lib/python3.8/dist-packages (from onnx==1.12.0) (3.20.1)
Requirement already satisfied: typing-extensions>=3.6.2.1 in /usr/local/lib/python3.8/dist-packages (from onnx==1.12.0) (4.1.1)
```

```
Successfully installed qubee-0.7.12
root@42fe959f7443: /workspace# python -c "import qubee; print(qubee.__version__)"
0.7.12
root@42fe959f7443: /workspace#
```

Step3-1. Make calibration set



Calibration Data

데이터 종류: Sample input들

저장 형식: .npy 파일



모델 컴파일시 float32->int8 양자화를 위해 사용



Calibration data 생성을 위한 util function을 제공하나,
util function을 사용하지 않아도 무방함

```
import torch
import numpy as np
from PIL import Image
import torchvision.transforms.functional as F
from qubee import make_calib_man

def preprocess_resnet50(img_path: str):
    img = Image.open(img_path)
    resize_size=(232, 232)
    crop_size=(224, 224)
    mean=[0.485, 0.456, 0.406]
    std=[0.229, 0.224, 0.225]
    out = F.pil_to_tensor(img)
    out = F.resize(out, size=resize_size)
    out = F.center_crop(out, output_size=crop_size)
    out = out.to(torch.float, copy=False) / 255.
    out = F.normalize(out, mean, std)
    out = np.transpose(out.numpy(), axes=[1, 2, 0])
    return out

if __name__ == "__main__":
    """
    The following code assumes that images to be used for calibration is prepared in
    directory `/workspace/sample/calibration/imagenet`.
    """
    make_calib_man(
        pre_ftn=preprocess_resnet50,
        data_dir="/workspace/sample/calibration/imagenet",
        save_dir="/workspace/sample/calibration",
        save_name="resnet50",
        max_size=50
    )
```

Calibration Data 생성 코드 예시

Step3-2. Compile Model

✔ Qubee에는 mxq_compile이라는 high-level api를 제공

```
from qubee import mxq_compile

if __name__ == "__main__":
    """
    The following code assumes that calibration data and the model are prepared in
    directory '/workspace/sample/calibration/resnet50' and
    '/workspace/sample/models/resnet50.onnx', respectively.
    """
    mxq_compile(
        model="/workspace/sample/models/resnet50.onnx",
        calib_data_path="/workspace/sample/calibration/resnet50",
        quantize_method="maxpercentile"
    )
```

✔ Input information과 output information 확인

컴파일 코드 예시

✔ Compilation was successful이라는 문구가 나오면 컴파일 완료

```
===== input information =====
input name      = input
input Shape     = (224, 224, 3)
reshaped Shape  = (224, 224, 3)
=====

===== output information =====
output name     = output
output Shape    = (1, 1, 1000)
=====

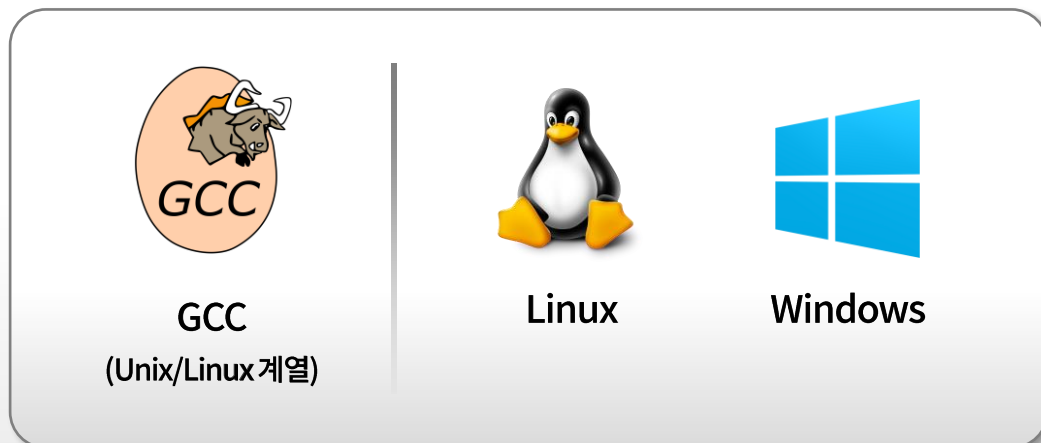
Try to load calibration data set from dataset.txt. Dataset t
There is 11 files in txt file.
Complete load calibration data. Success = 11, Fail = 0
Load calibration data Time = 0.0246369 sec
[[[B][[C[2023-09-12 17:31:06.467] Compiling the Quantized m
[2023-09-12 17:31:06.469] Compile scheme: single
[2023-09-12 17:31:06.469] Optimize num divisions
[2023-09-12 17:31:06.470] Find force division
[2023-09-12 17:31:07.213] Optimization finished in 0.746545
[2023-09-12 17:31:07.268] re_opt Loss: 38455360, mac: 459318
[2023-09-12 17:31:07.281] optimization finished
[2023-09-12 17:31:07.281] SAVE DIV/FDIV JSON FOR DEBUG
[2023-09-12 17:31:07.281] called: implement_division
[2023-09-12 17:31:13.206] called: implement_division
[2023-09-12 17:31:18.358] called: implement_division
[[[C][[B][[B[2023-09-12 17:31:23.424] called: implement_divi
[2023-09-12 17:31:28.513] called: implement_division
[2023-09-12 17:31:33.931] called: implement_division
[2023-09-12 17:31:39.109] called: implement_division
[2023-09-12 17:31:44.294] called: implement_division
[2023-09-12 17:31:49.253] called: implement_division
[2023-09-12 17:31:49.265] Elaborating MXQ...
[2023-09-12 17:31:49.271] Exporting MXQ...
[2023-09-12 17:31:49.304] Verifying exported MXQ file...
[2023-09-12 17:31:49.323] Compilation was successful.
root@42fe959f7443:/workspace/samplecode#
```

컴파일 출력문

Environment Setup (Runtime Library)

C++ 언어 지원

- ✓ Linux Kernel 계열 및 Windows 운영체제 모두 지원
- ✓ GCC 계열 (Linux, Windows), MSVC 계열 (Windows) 컴파일 지원



Python 언어 지원

- ✓ Linux Kernel 계열 운영체제 지원



Model Inference Code 예시

```
#include "maccel/maccel.h"
#include "maccel/model.h"

using mobilint

StatusCode sc;
std::unique_ptr<Accelerator> acc = Accelerator::create(sc);

ModelConfig mc;
mc.excludeAllCores();
mc.include(Cluster::Cluster0, Core::Core0);
mc.include(Cluster::Cluster1, Core::Core0);

std::unique_ptr<Model> model = Model::create(mxq_path, mc, sc);
model->launch(*acc.get());

vector<vector<float>> result = model->infer(input, sc);
```

C++ Inference 코드 예시

라이브러리 호출

NPU Class 생성

(옵션)NPU 코어 지정

Model Class 생성

Model을 NPU에 탑재

Model Inference

```
import maccel
mxq_path = "./mxqfiles/test.mxq"

acc = maccel.Accelerator()
mc = maccel.ModelConfig()
mc.exclude_all_cores()
mc.include(maccel.Cluster.Cluster0, maccel.Core.Core0)
mc.include(maccel.Cluster.Cluster1, maccel.Core.Core0)

model = maccel.Model(mxq_path, mc)
model.launch(acc)

### Inference
result = model.infer(input list)
```

Python Inference 코드 예시

Model Coverage – Supported Layers (NPU)

Layers	Comments
Convolution	
Depthwise Convolution	
Group Convolution	
Transpose Convolution	
Gemm	Only for TransA = 0
Batchnorm	Inference mode
Maxpool	Dilation = 1
Avgpool	- Dilation = 1 - Include_pad = True
ReduceMean	Along height and width
ReduceMax	Along height and width
Global Average Pool	
Resize	- Nearest, bilinear(align_corner = False) - All support downsample, upsample
Add	Broadcast only available for constant
Sub	Broadcast only available for constant
Mul	- Only constant multiplication - Layerwise will be supported in next version chip
Div	- Only constant multiplication - Layerwise will be supported in next version chip

Model Coverage – Supported Layers (NPU)

Layers	Comments
Concatenate	
Relu	
Prelu	Only scalar slope
LeakyRelu	
Sigmoid	
Tanh	
Softplus	
HardSwish	
HardSigmoid	
Swish	
Mish	
Clip	
Exp	
Slice	
Reshape	Only channel-wise flatten and before fully connected layer or conv layer
Transpose	- Only for special cases - Ex) Transpose-Flatten-Linear, Reshape-Transpose-Reshape

Model Coverage – Supported Layers (NPU)

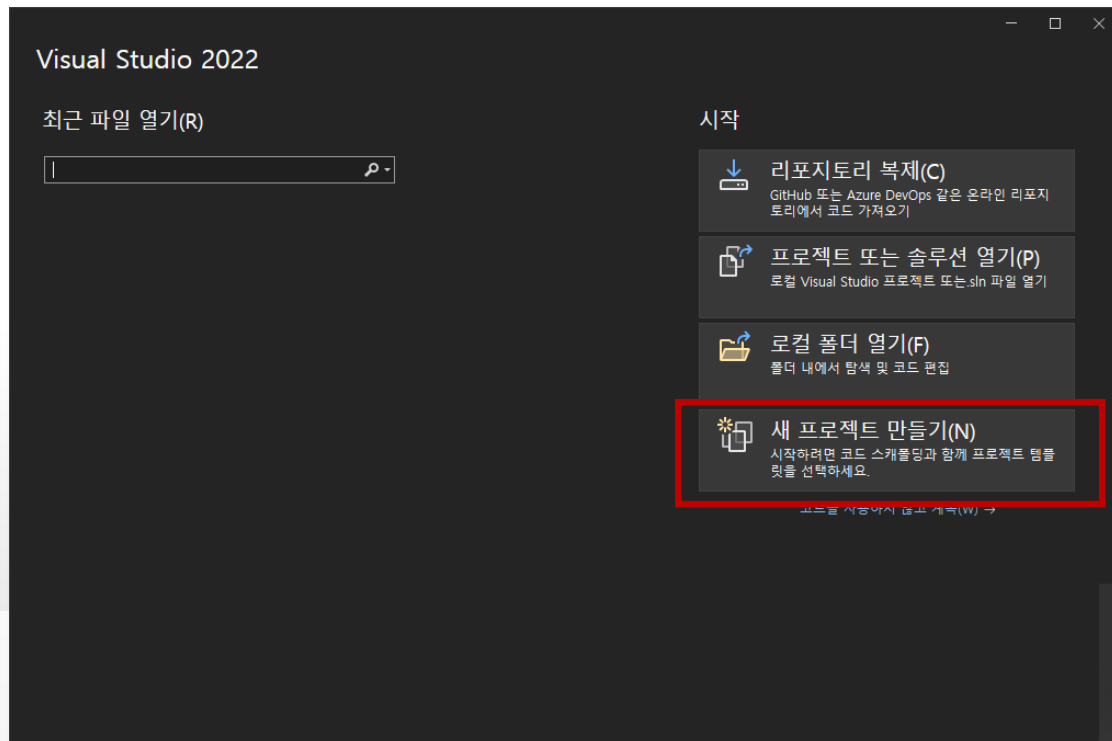
Layers	Comments
Squeeze	Only for special cases
Flatten	Only axis = 1 and before fully connected layer or Conv
Tile	Batch-wise tile is unsupported
Constant	
Shape	
ConstantOfShape	
Identity	
Pad	
Gelu	
Pow	
Elu	
Log	
addPartial	

 High-level Compile, Low-level Compile 단계에서 서로 협업하여 지원하는 Layer들을 지속적으로 추가 중이며,
 Layer의 Constraints를 해소하기 위한 추가기능 or 우회방법 구현 중

Microsoft Visual Studio 개발 환경 설정

✓ 솔루션 파일 생성 후 Header Path 및 Library Path 설정 후 개발 진행 가능

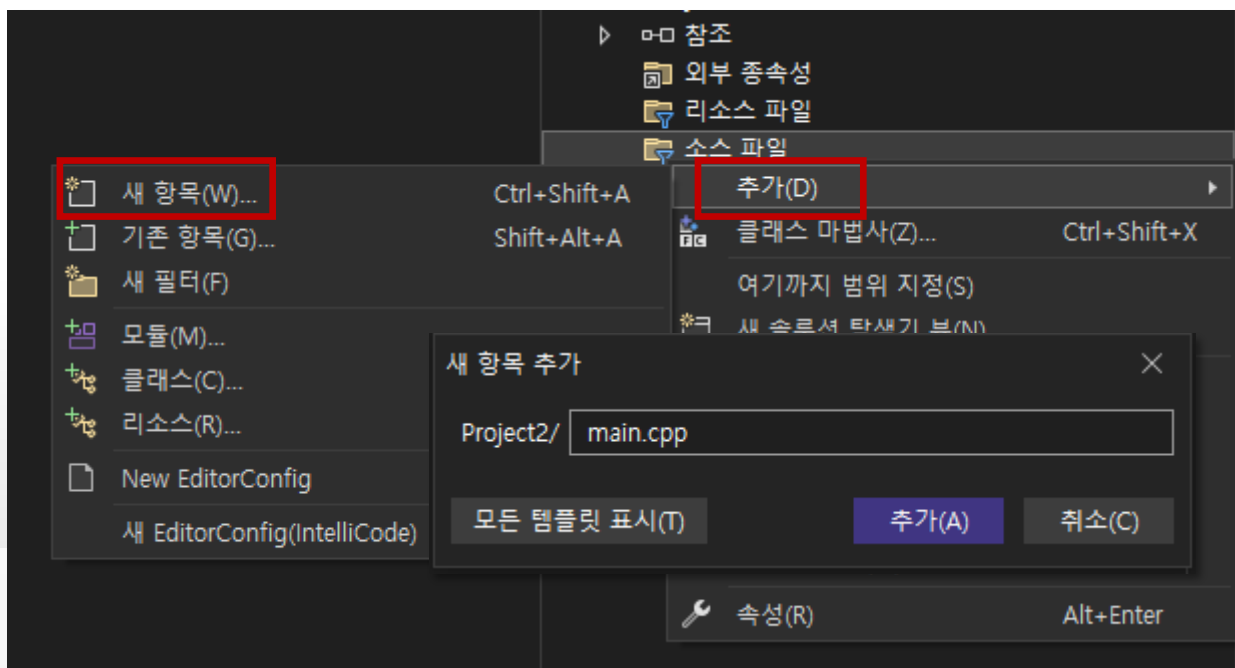
새로운 프로젝트를 생성합니다.



Microsoft Visual Studio 개발 환경 설정

- ✓ 솔루션 파일 생성 후 Header Path 및 Library Path 설정 후 개발 진행 가능

새로 시작할 신규 소스코드를 하나 작성합니다.
빨간 줄이 보이는데 이는 뒤에 이어지는 과정으로 해결합니다.



```
#include <iostream>

#include "maccel/maccel.h"

using namespace mobilint;

int main() {
    StatusCode sc;
    std::unique_ptr<Accelerator> acc = Accelerator::create(sc);
    std::unique_ptr<Model> model = Model::create("resnet50.mxq", sc);

    auto mi = model->getModelInputShape()[0];
    int h = mi[0];
    int w = mi[1];
    int c = mi[2];

    auto input_img = std::make_unique<float[]>(h * w * c);

    model->launch(+acc);

    int i = 0;
    while (true) {
        std::cout << i++ << std::endl;
        auto result = model->infer({ input_img.get() }, sc);
        if (i > 100) {
            break;
        }
    }

    model->dispose();
    return 0;
}
```

Microsoft Visual Studio 개발 환경 설정

- ✓ 솔루션 파일 생성 후 Header Path 및 Library Path 설정 후 개발 진행 가능

프로젝트에 배포받은 Runtime Library (include, lib 폴더) 를 복사합니다.

maccel_mobilint_aries_v0.17.0 > maccel	
이름	수정한 날짜
bin	2024-04-12 오전 4:44
include	2024-04-12 오전 4:44
lib	2024-04-12 오전 4:44
python	2024-04-12 오전 4:44



test > test	
이름	수정한 날짜
include	2024-07-31 오전 6:27
lib	2024-07-31 오전 6:27
main.cpp	2024-07-31 오전 5:56
test.vcxproj	2024-07-31 오전 6:23
test.vcxproj.filters	2024-07-31 오전 6:23
test.vcxproj.user	2024-07-31 오전 5:54

Microsoft Visual Studio 개발 환경 설정

✓ 솔루션 파일 생성 후 Header Path 및 Library Path 설정 후 개발 진행 가능

배포받은 Runtime Library의 bin 폴더에 있는 DLL 파일 및 모델 파일(MXQ)도 같은 폴더에 복사합니다.

maccel_mobilint_aries_v0.17.0 > maccel > bin	
이름	수정한 날짜
maccel.dll	2024-04-12 오전 4:44
maccel.d.dll	2024-04-12 오전 4:42

maccel_mobilint_aries_v0.17.0 > resnet50	
이름	수정한 날짜
ILSVRC2012_val_00000001.JPEG	2024-01-31 오전 8:47
maccel.dll	2024-04-12 오전 4:44
maccel.d.dll	2024-04-12 오전 4:42
resnet50.cpp	2024-01-31 오전 8:47
resnet50.mxq	2024-01-31 오전 8:47
resnet50.py	2024-01-31 오전 8:47

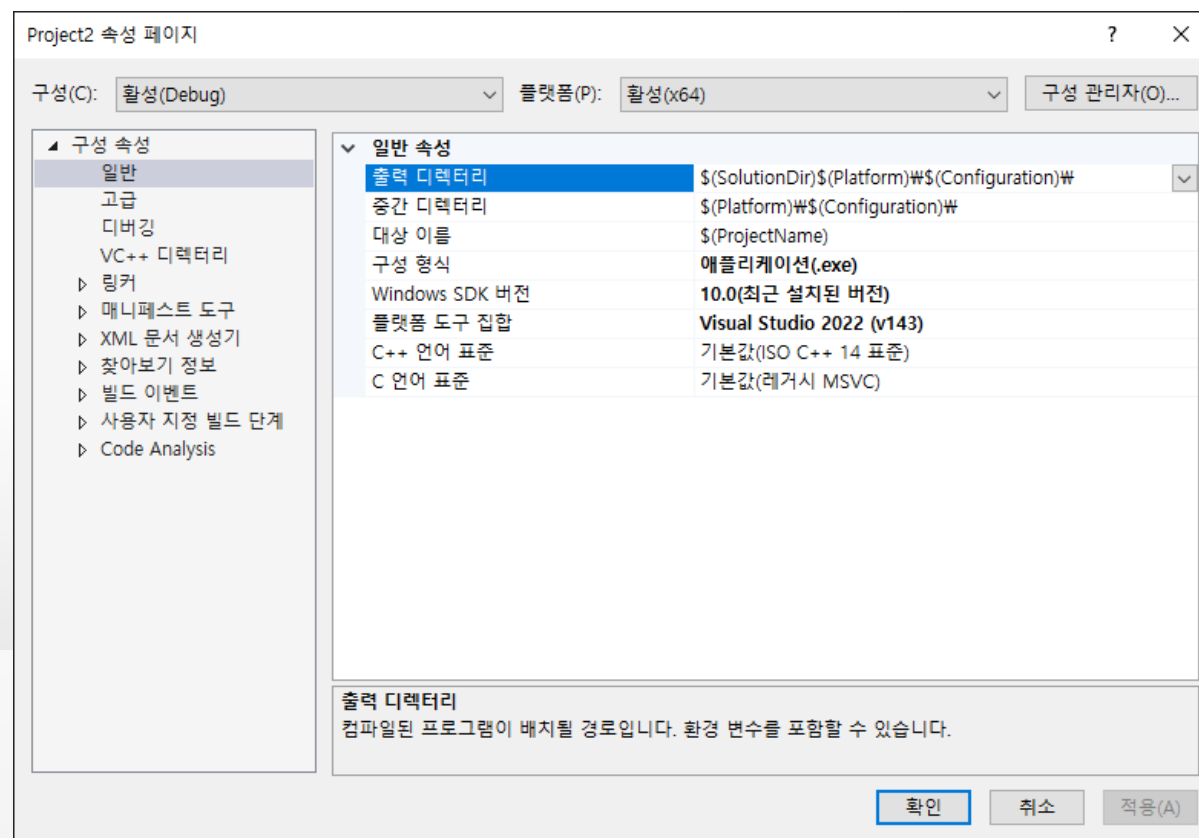
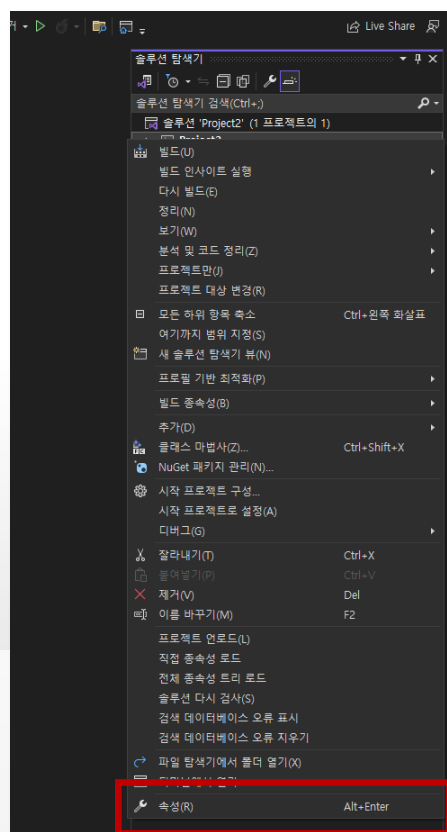


test > test	
이름	수정한 날짜
include	2024-07-31 오전 6:27
lib	2024-07-31 오전 6:27
maccel.dll	2024-04-12 오전 4:44
maccel.d.dll	2024-04-12 오전 4:42
main.cpp	2024-07-31 오전 5:56
resnet50.mxq	2024-01-31 오전 8:47
test.vcxproj	2024-07-31 오전 6:23
test.vcxproj.filters	2024-07-31 오전 6:23
test.vcxproj.user	2024-07-31 오전 5:54

Microsoft Visual Studio 개발 환경 설정

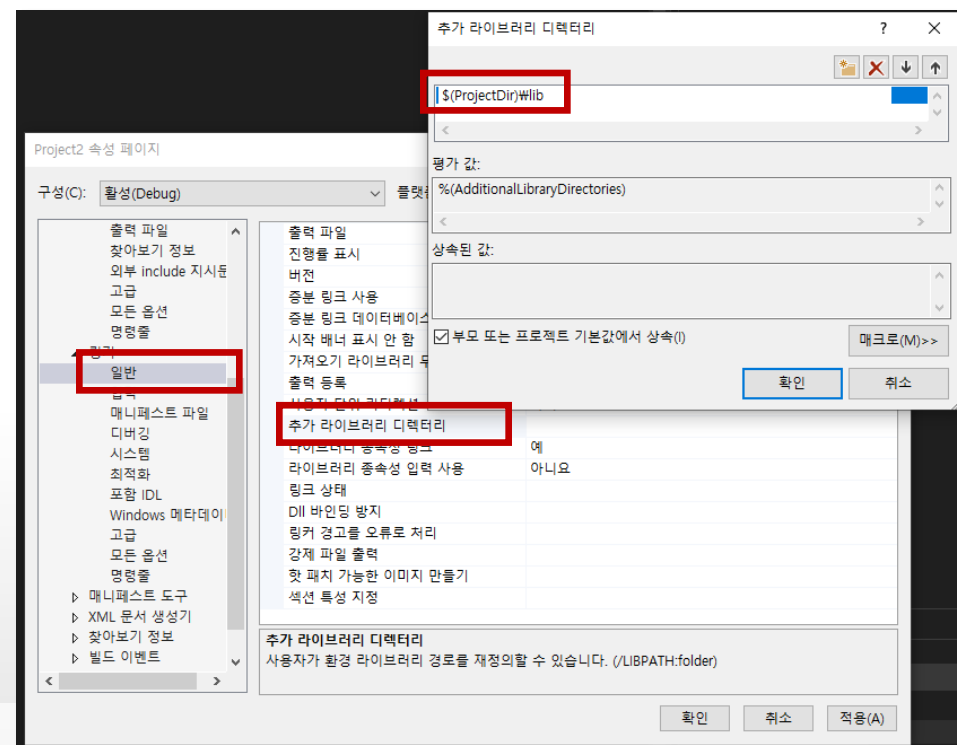
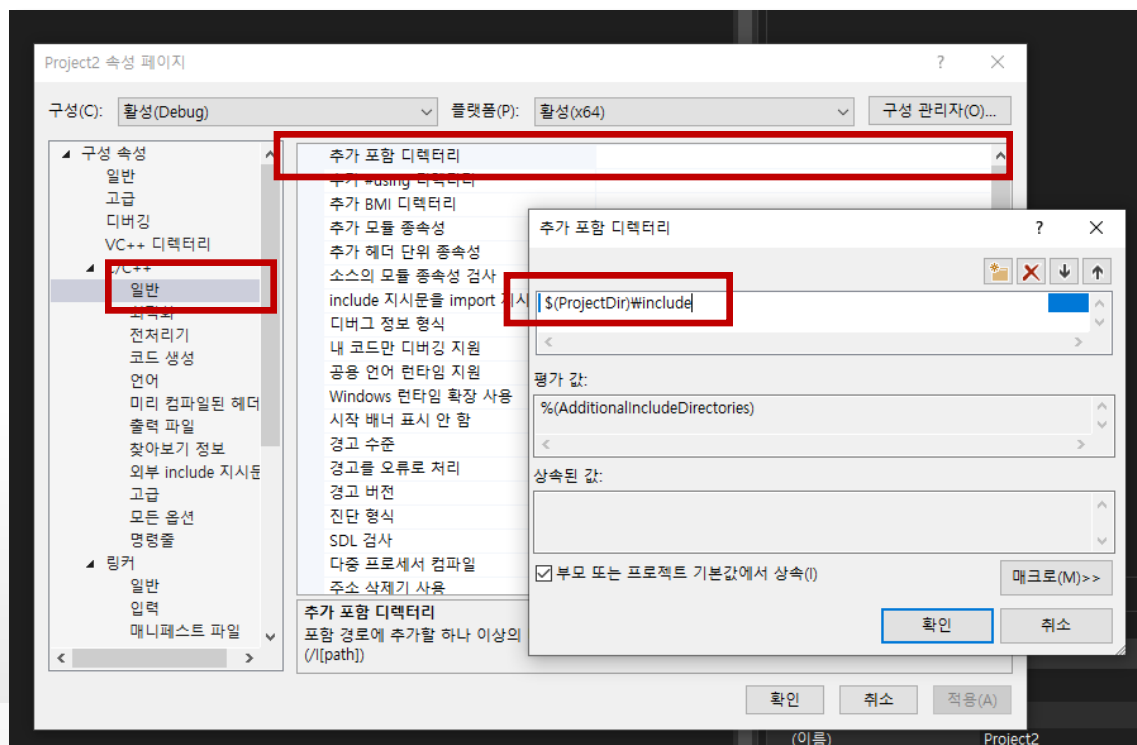
✓ 솔루션 파일 생성 후 Header Path 및 Library Path 설정 후 개발 진행 가능

여기부터는 Header와 Library Path 를 수정합니다.



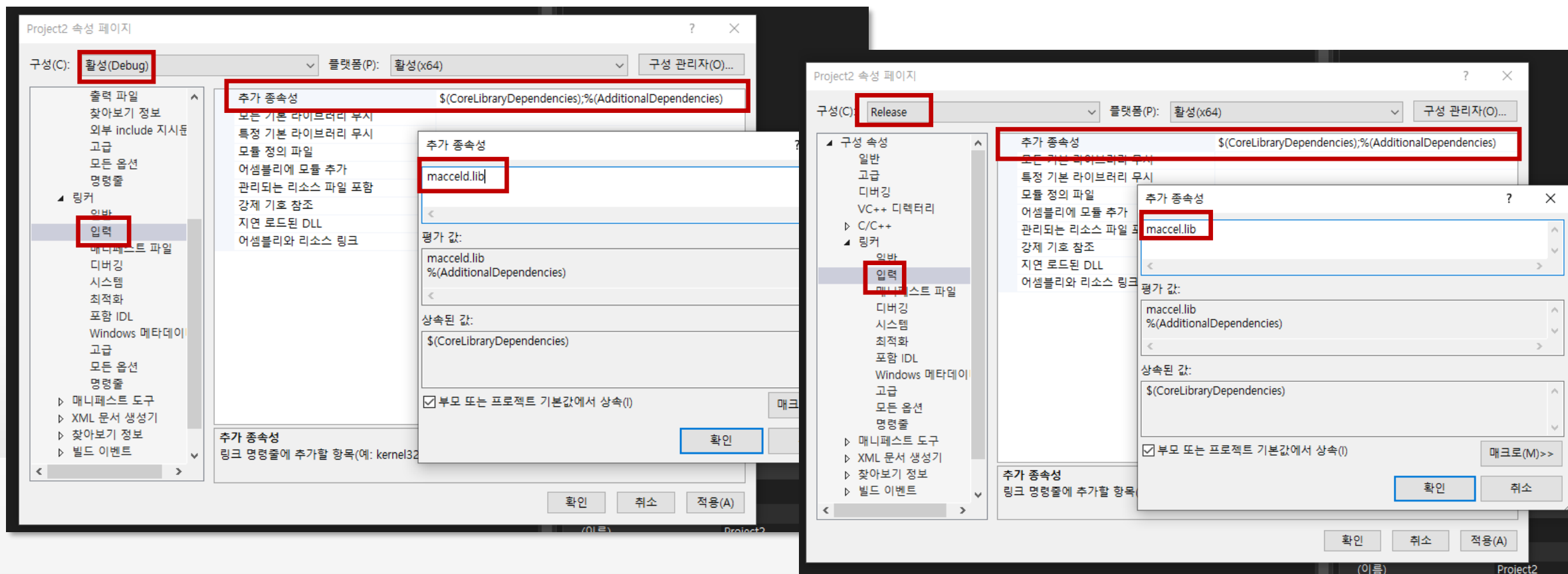
Microsoft Visual Studio 개발 환경 설정

✓ 솔루션 파일 생성 후 Header Path 및 Library Path 설정 후 개발 진행 가능

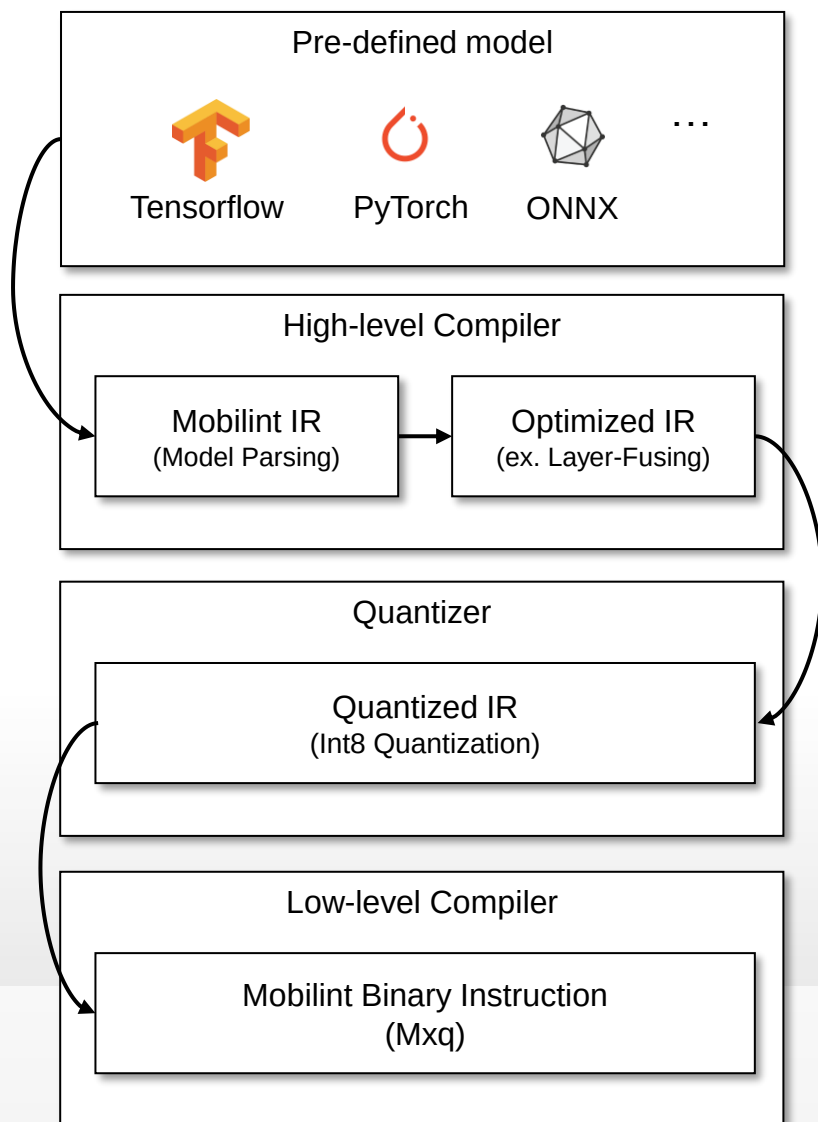


Microsoft Visual Studio 개발 환경 설정

- ☑ 참고: 빌드 설정 (Debug/Release) 마다 각기 설정이 있으므로 양 설정 모두를 앞선 내용대로 변경 요망
 maccel 은 Release, macceld 는 Debug 용이며 잘못 지정 시에는 실행 불가함 (e.g. Debug 빌드에 maccel.lib 사용)



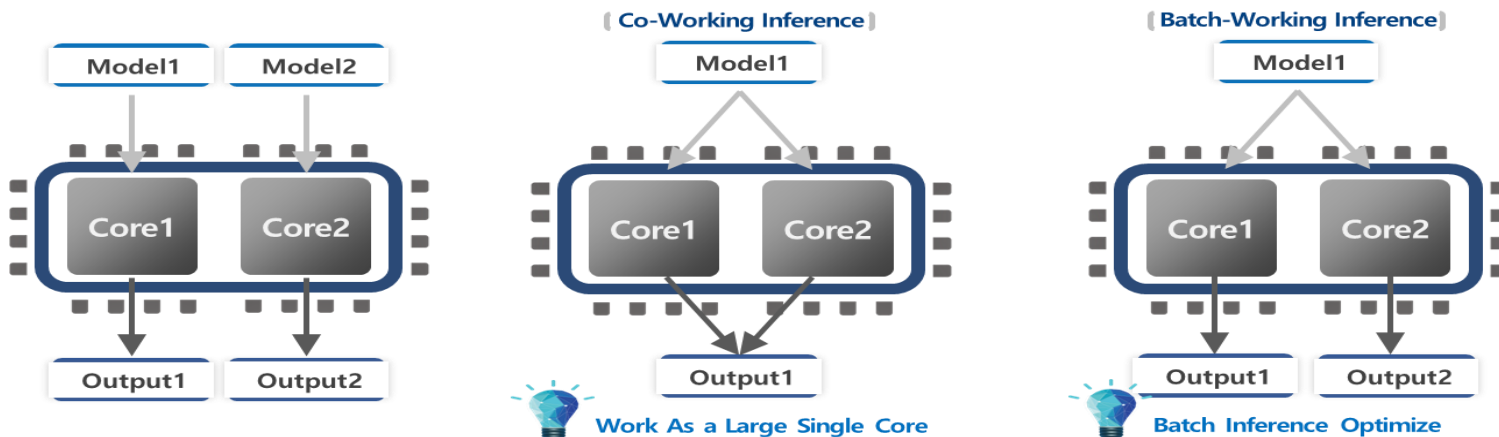
[별첨] Compiler Structure



- ✓ Mobilint IR을 통해 여러 Deep Learning Framework 지원 가능
- ✓ High-level Optimizing(Layer Fusing, Layer Rearrangement 등)을 적용하여 연산 성능 최적화
- ✓ 자체 개발한 Quantizer를 이용하여 NPU 구조에 최적화된 Quantization 결과 제공
- ✓ Target Hardware에 맞추어 데이터 이동, 연산 순서 등을 최적화하여 자체제작한 Binary Instruction set으로 변환

[별첨2] Aries2 추가 기능

✓ Global/Multi Core Scenario



✓ Transformer 지원

✓ Int16 Inference 지원 및 Mixed-Precision 지원

✓ Float Input Direct하게 지원

✓ Host-NPU 통신 프로토콜 개선